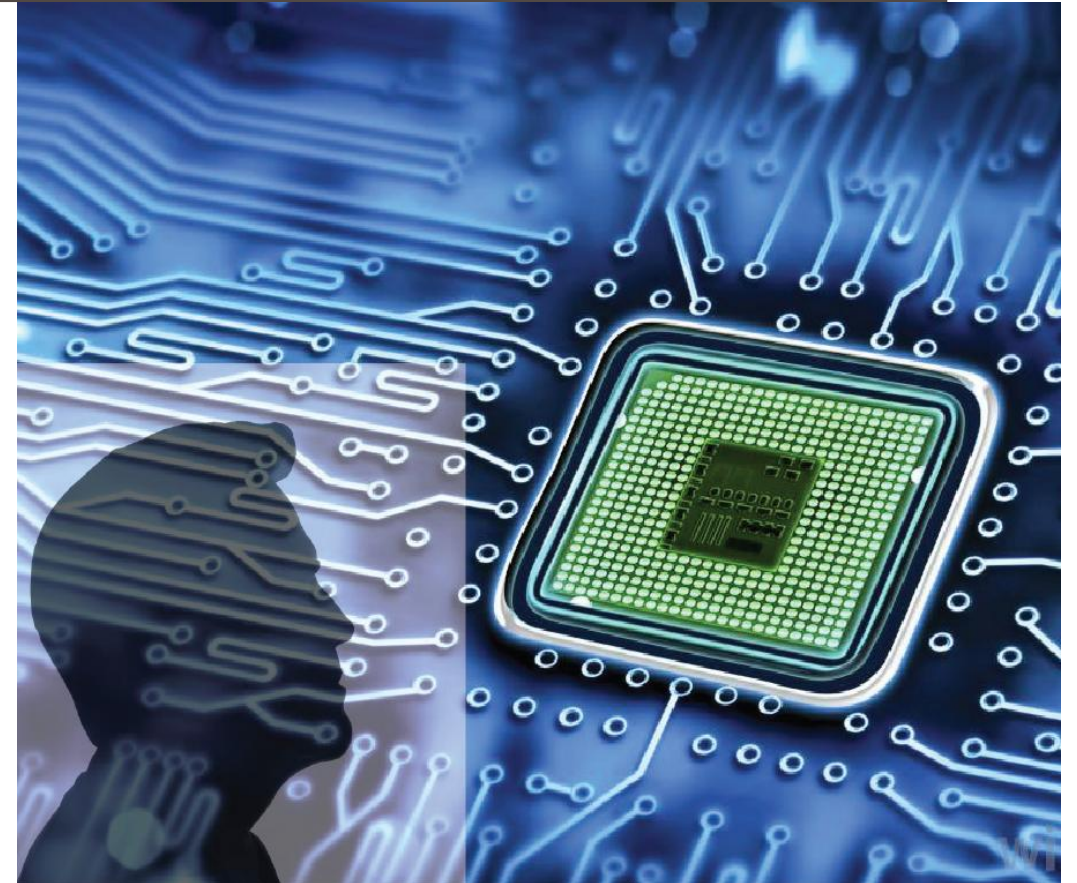


ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-1, Lecture-2 Introduction

Sneh Saurabh
14th August, 2025



Electronic System Level (ESL) Design

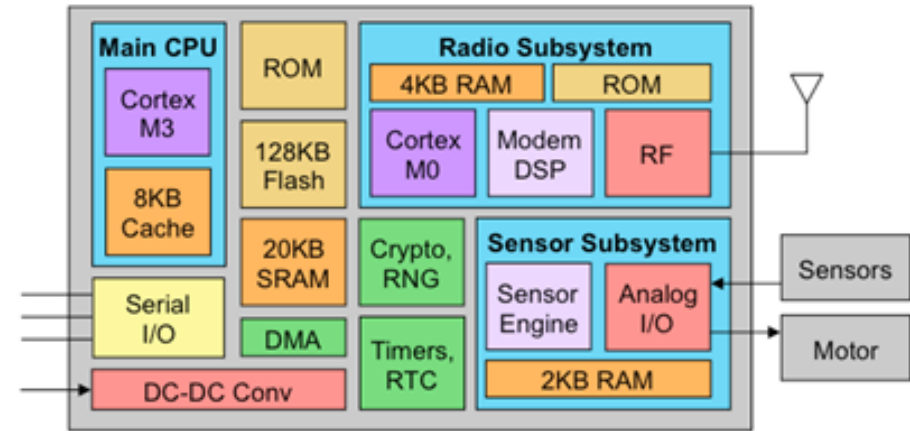
Motivation

Challenges in SoC design:

- Increased complexity
 - More features
 - More components (hardware, software), more interactions/communication
 - Deciding architecture of the system (of both HW and SW) becomes more challenging

Effect of increased complexity:

- Time taken to design increases
 - Strict constraints on Time to Market
- Increased resource for designing
 - Increase Cost
- Compromise on FoMs
 - Constraints from specification



- Block diagram of a low-power TI CC26xx processor.
- Source: <https://semiengineering.com/soc-integration-complexity-size-doesnt-always-matter/>

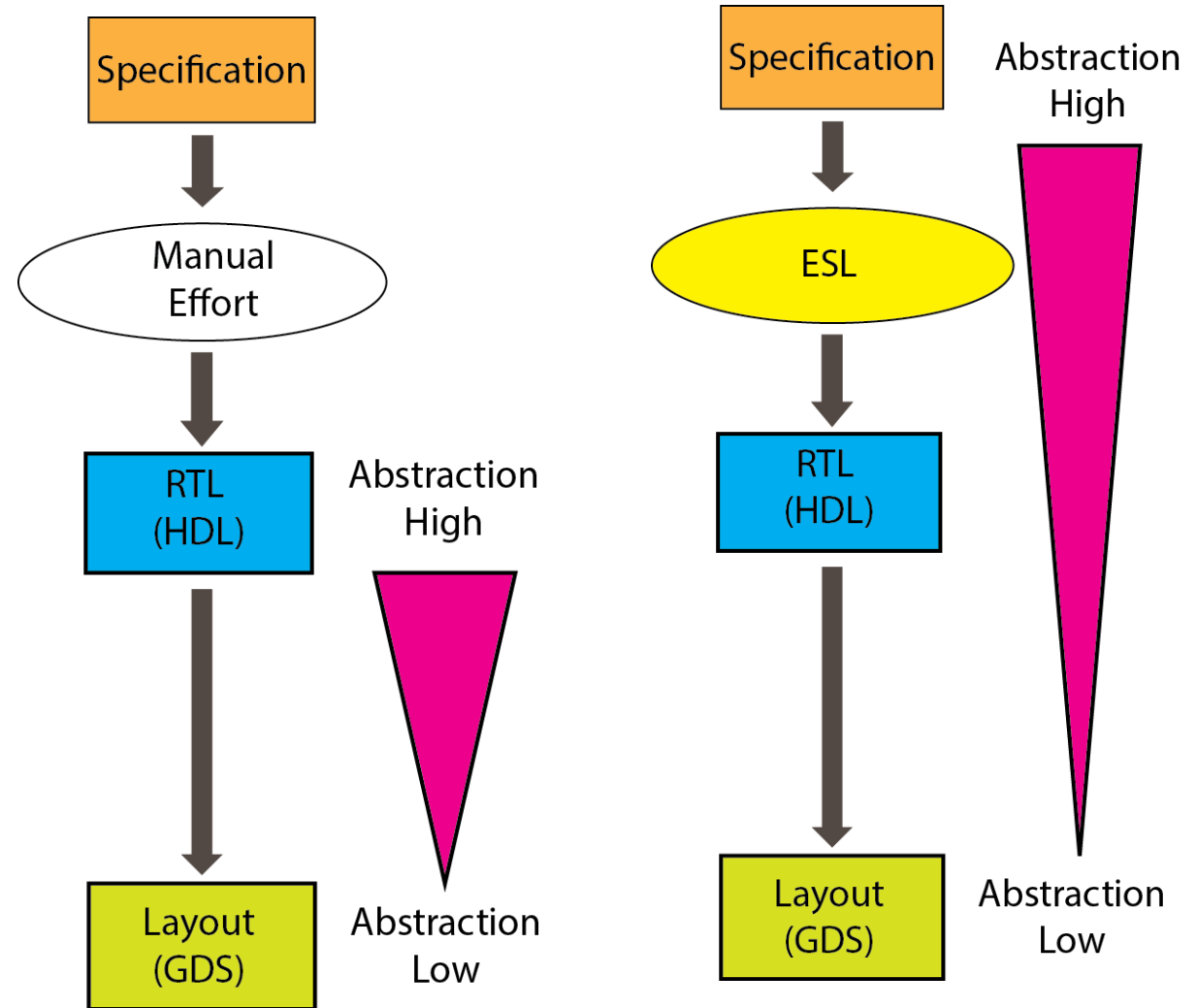
Solution

- Increase the design process efficiency
 - Move to a higher-level of abstraction
 - ESL as a solution

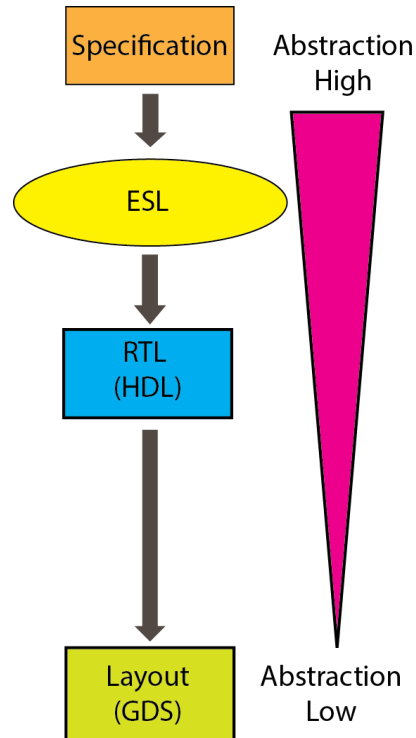
What is ESL?

Definition of ESL:

- **Higher** level abstraction of a design that enables **better** comprehension of a system
 - Higher and better compared to the traditional abstraction level such as RTL
-
- **Expectation:** overall time taken to design decreases, FoM improves and is cost-effective



ESL: Decisions and Deliverables

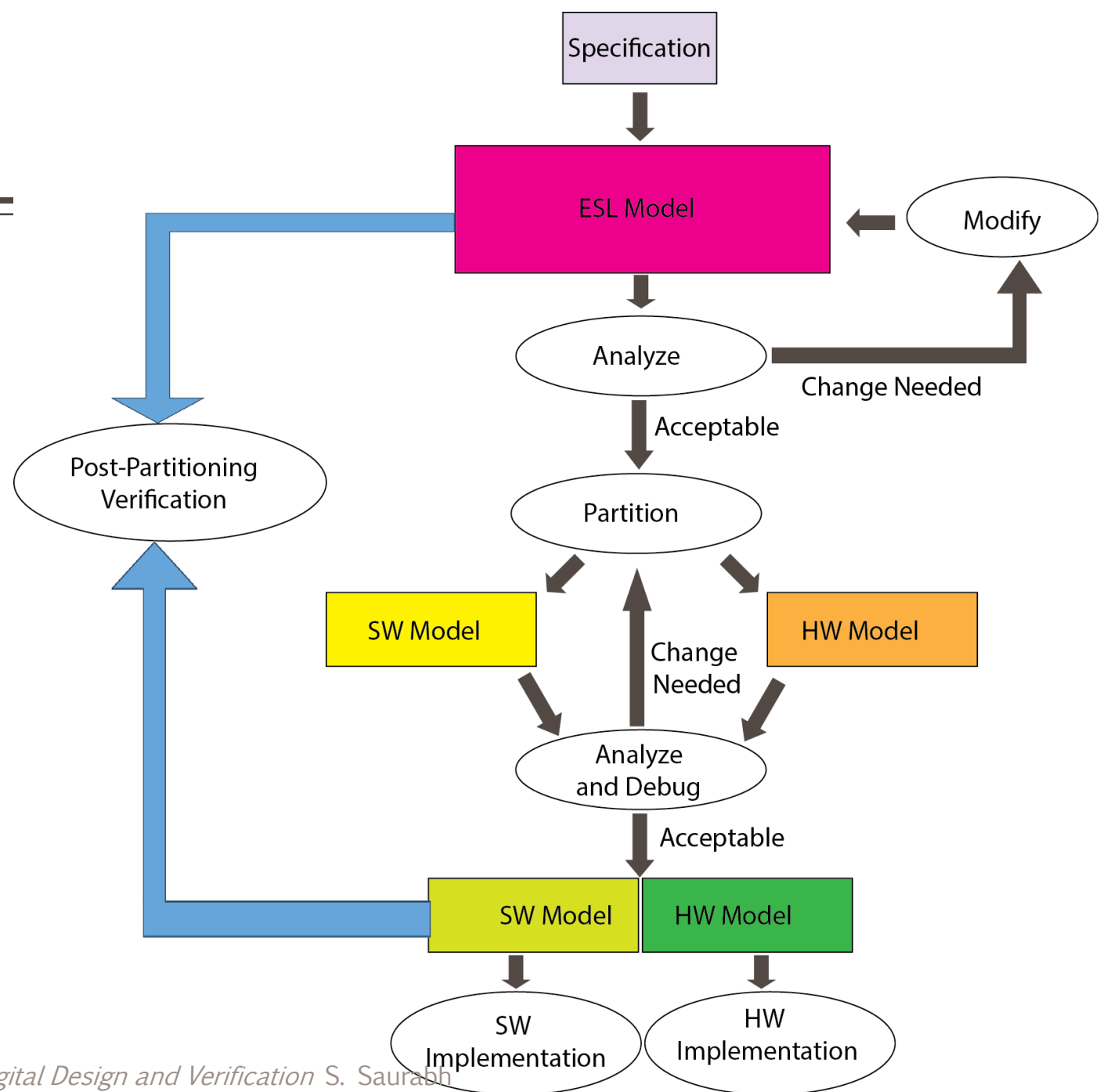


Architectural exploration, estimation and optimization:

- **Concurrency:** sequential, parallel (multiple invocation point), multithreaded (common invocation common point), pipelining, etc.
 - **Communication:** bus-based, buffered, memory, point to point, etc.
 - **Functionality:**
 - Is current algorithm sufficient? Is bus architecture and arbitration scheme acceptable? Will data representation (fixed point, or floating point) work?
-
- **HW-SW partitioning:** with less experimentation
 - **Enable parallel software development:** before hardware prototype availability
 - **Enable IP and HW-SW integration**
 - **RTL generation**
 - **Enable faster and more exhaustive verification:**
 - HW and SW co-verification (mixing various levels of abstraction)
 - Generate Testbench for RTL verification

ESL: Flow

- ESL implementations can be done in many ways
- One of the ESL flow shown alongside



Specification and ESL Model (1)

- Developing documents that describe system or product intent and constraints

- Top-level specification can be in Natural Language

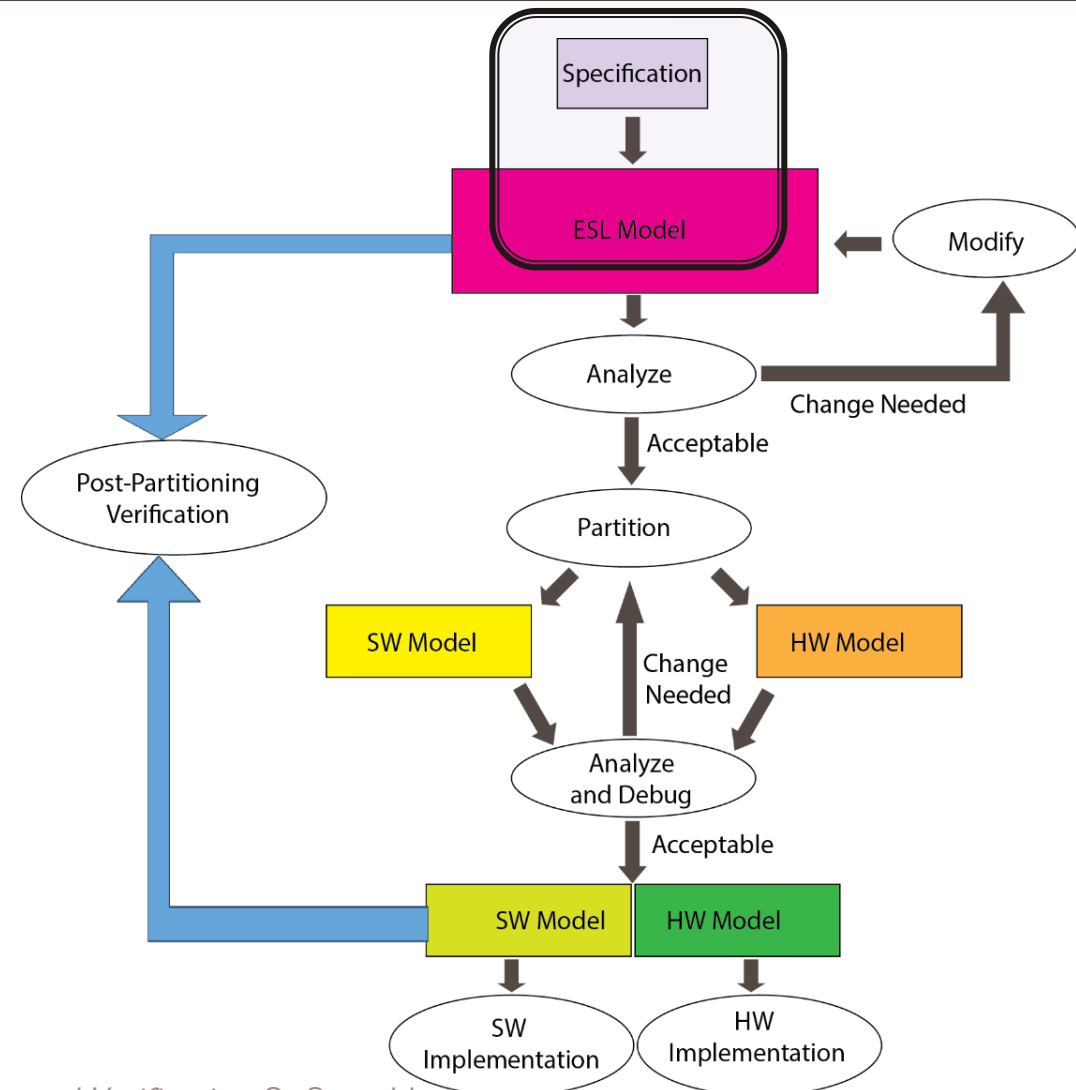
Executable specification:

- A functional or behavioral description of a system that when run as a computer program, exhibits the intended behavior

Specification Description:

- MATLAB, SystemC, TLM, SystemVerilog
- Extensible Markup Language (XML), IP-XACT, Specification and Description Language (SDL), etc.

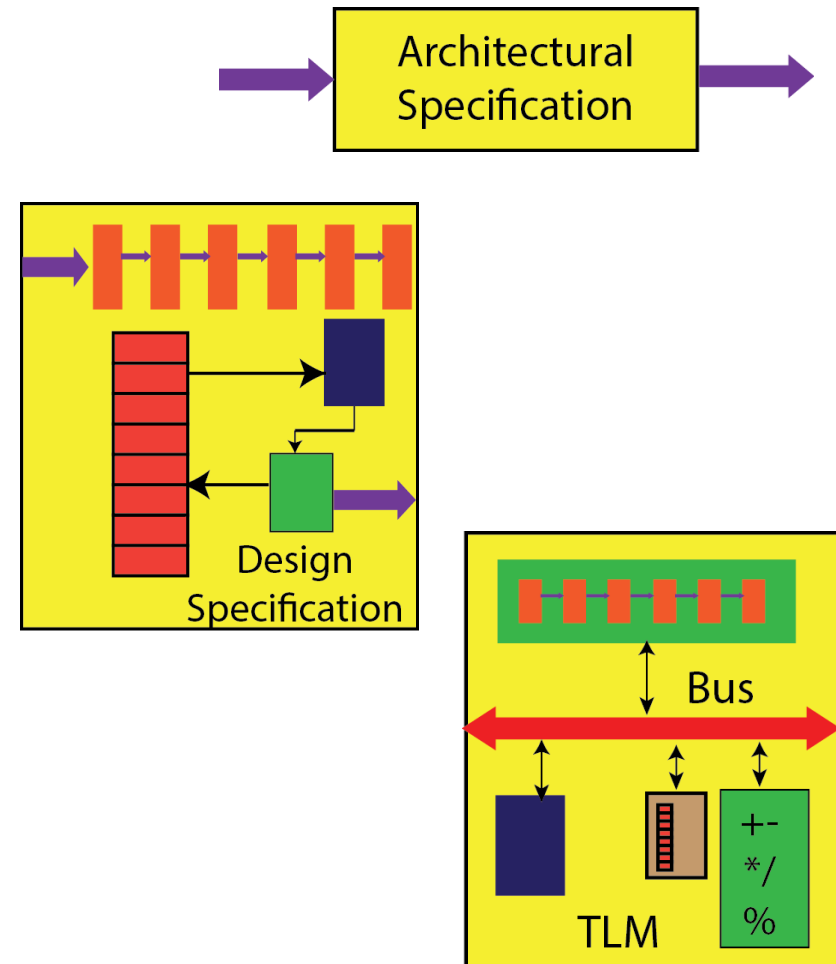
We will discuss SystemC, TLM and IP-XACT in more detail



Specification and ESL Model (2)

Executable specification can be at various abstraction levels

- **Architecture specification:** reflect functionality or behavior of system
 - Example: what instructions exist for a processor and what these instructions do
 - Enables architectural exploration
- **Design specification:** reflects microarchitecture decisions
 - Example: pipeline length and layout, number and size of cache, cycle count for each instruction
- **Transaction-level modeling (TLM) :** reflects how data transferred between various components
 - Can be highly efficient compared to RTL in simulation

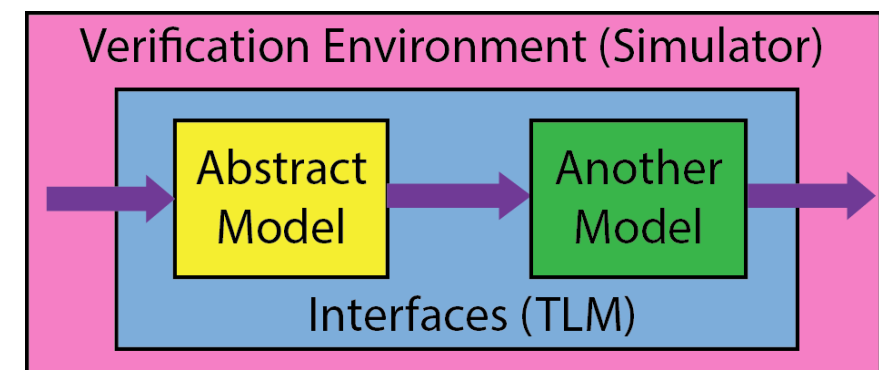
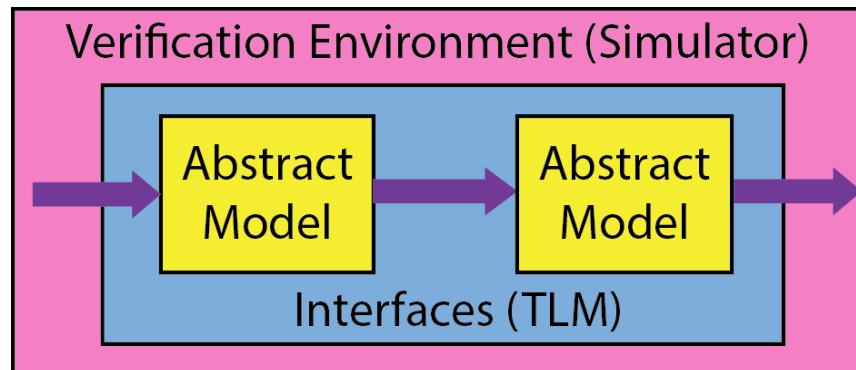


Specification and ESL Model (3)

Desirable Attributes of a Specification:

- Define functional requirements and constraints completely and precisely
- Avoid details that can constrain the implementation (try to make it platform independent)
- Act as a source for:
 - Functional Requirements
 - Design implementation
 - Verification

- Specification undergoes changes during the design flow:
 - Add details and refinement
- Expected tool behavior (example, simulator):
 - Should work at different level of abstractions and allow mixing of abstractions (instead of re-specifying)



ESL Model: Analysis and Modifications

Pre-partition Analysis

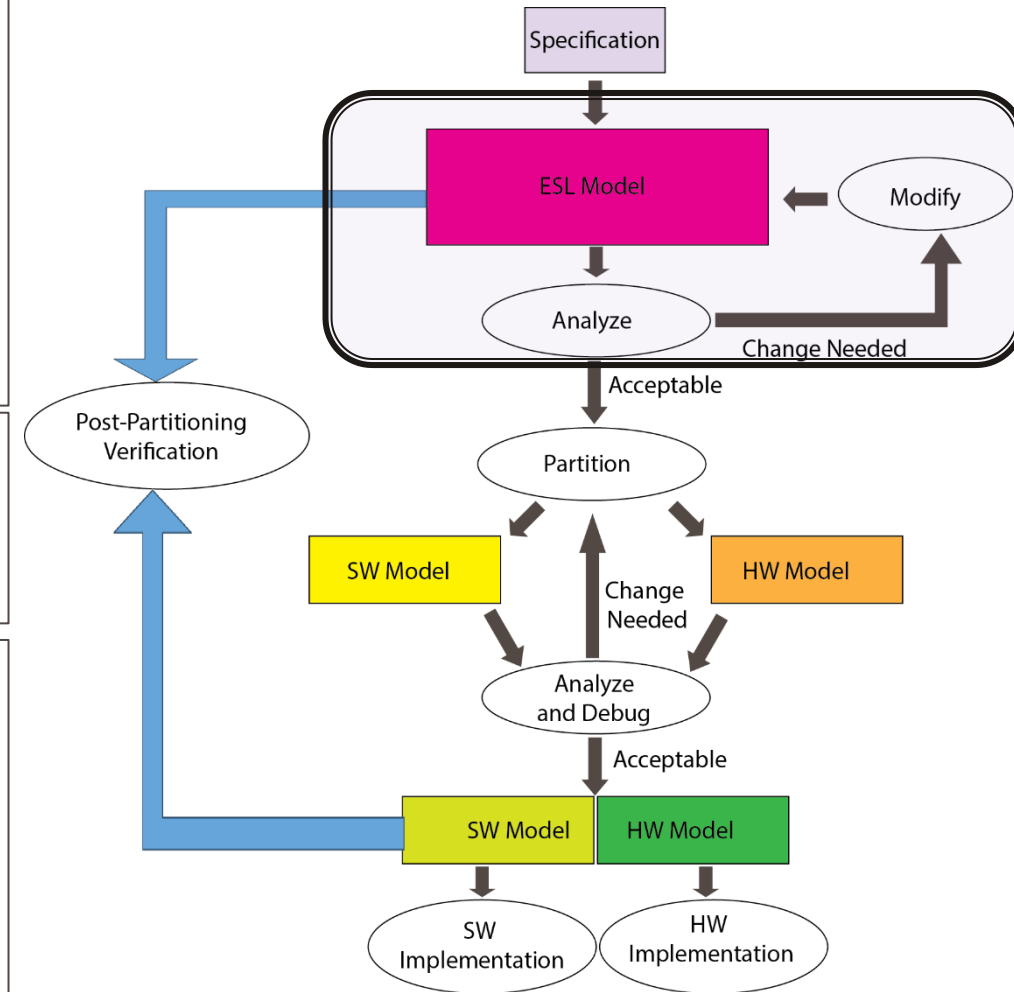
- Exploring the algorithmic tradeoffs: time, space, power, complexity, and time-to market domains
 - Some algorithms can be parallelized easily
 - Storage requirement of algorithms can differ
 - Power/Energy: speculative computing, caching

Static Analysis

- **Feasibility Analysis:** For non-executable specification check for maintainability, usability, criticality, and reliability.

Dynamic Analysis:

- Time-based performance: latency, throughput
- Impact of scheduling, arbitration among components
- Requirements on:
 - Computation: floating point, fixed point
 - Communication: band width

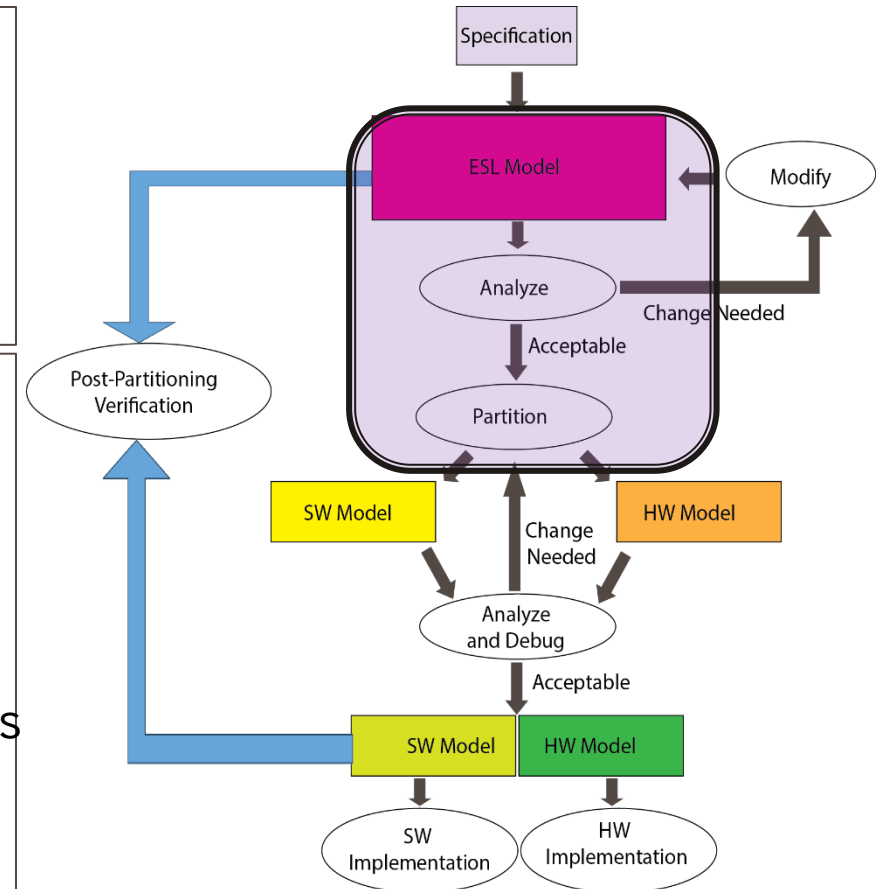


Partitioning

Partitioning

- The process of choosing which algorithms (or parts thereof) defined in the specification to implement as:
 - A. Software components running on processors
 - B. Hardware components
- Also need to decide:
 - Number and types of processors (possibly with changes in the instruction set)
 - Memory organization: ROM, SRAM, DRAM, etc.
 - Communication between them
- The division can be within software and hardware components also
 - Subdividing software among multiple processors
 - Subdividing hardware among different teams

We will discuss Partitioning in more detail



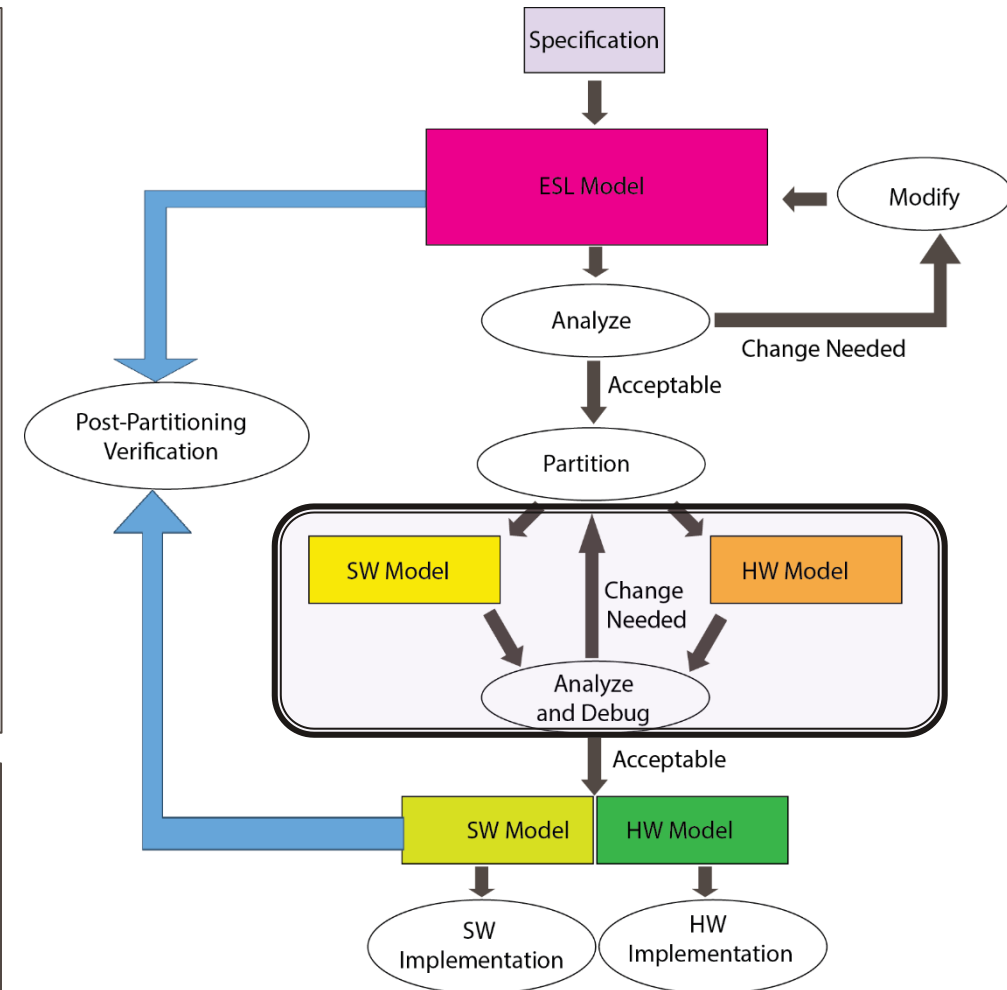
Post-partitioning Analysis and Debug

Post-partitioning Analysis and Debug

- Effects of hardware and software partitioning are examined
 - Algorithms running on the processor are implemented (C, C++, Java) and compiled/run on the target processor or Instruction Set Simulator (ISS)
 - Hardware is modeled behaviorally using SystemC, SystemVerilog, HDL
 - Interface should be compatible among SW and HW: they should execute together
- Change partition if needed: design space exploration

Types of Analysis

- Performance analysis, interface analysis, power analysis, area analysis, cost analysis, debug capability analysis



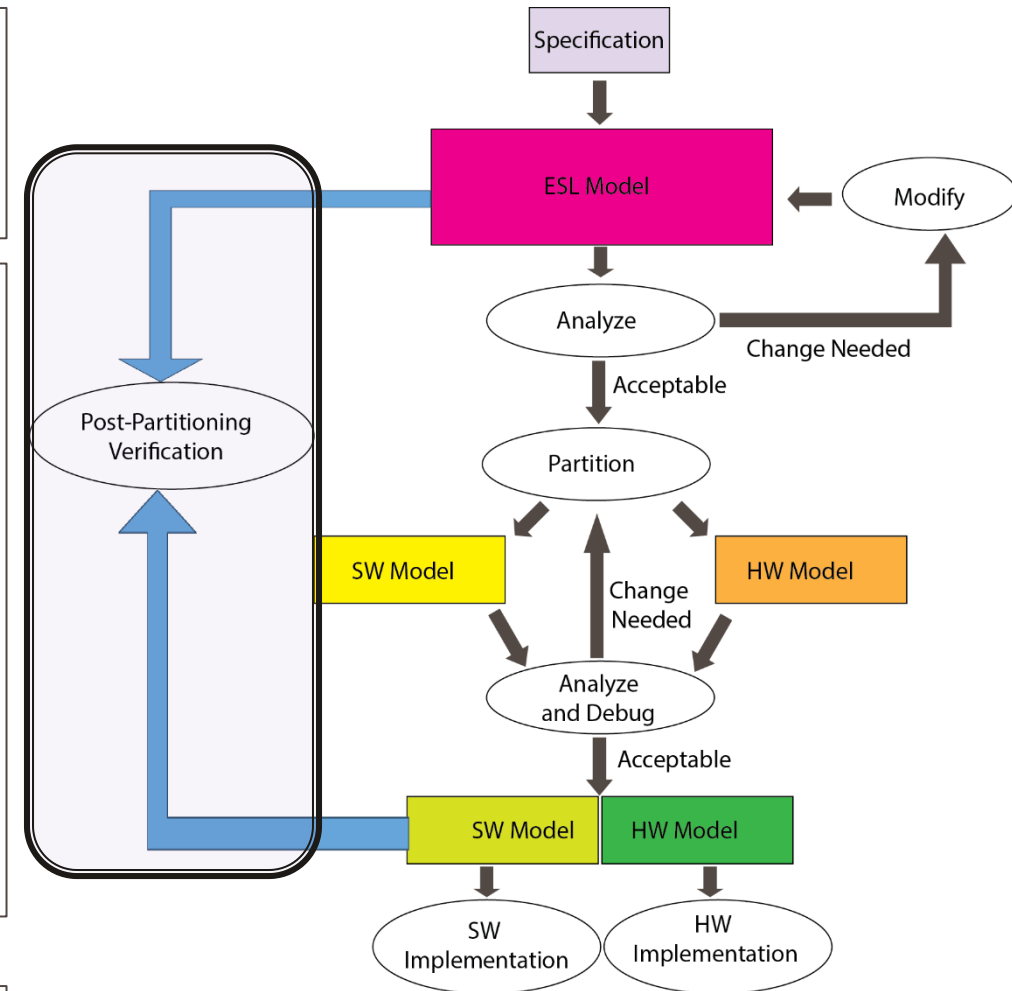
Post-partitioning Verification

Post-partitioning Verification

- Ensure that the intended behavior is preserved by the post-partitioned models

Tasks involved

- Verification planning
 - Coverage model: at a higher level of abstraction than RTL (for example, TLM)
 - Properties (assertions): at this level of abstraction
- Verification environment (infrastructure) implementation
 - SystemVerilog models for coverage, assertion, and infrastructure are widely used.
- Analyzing results



We will discuss SystemVerilog in more detail

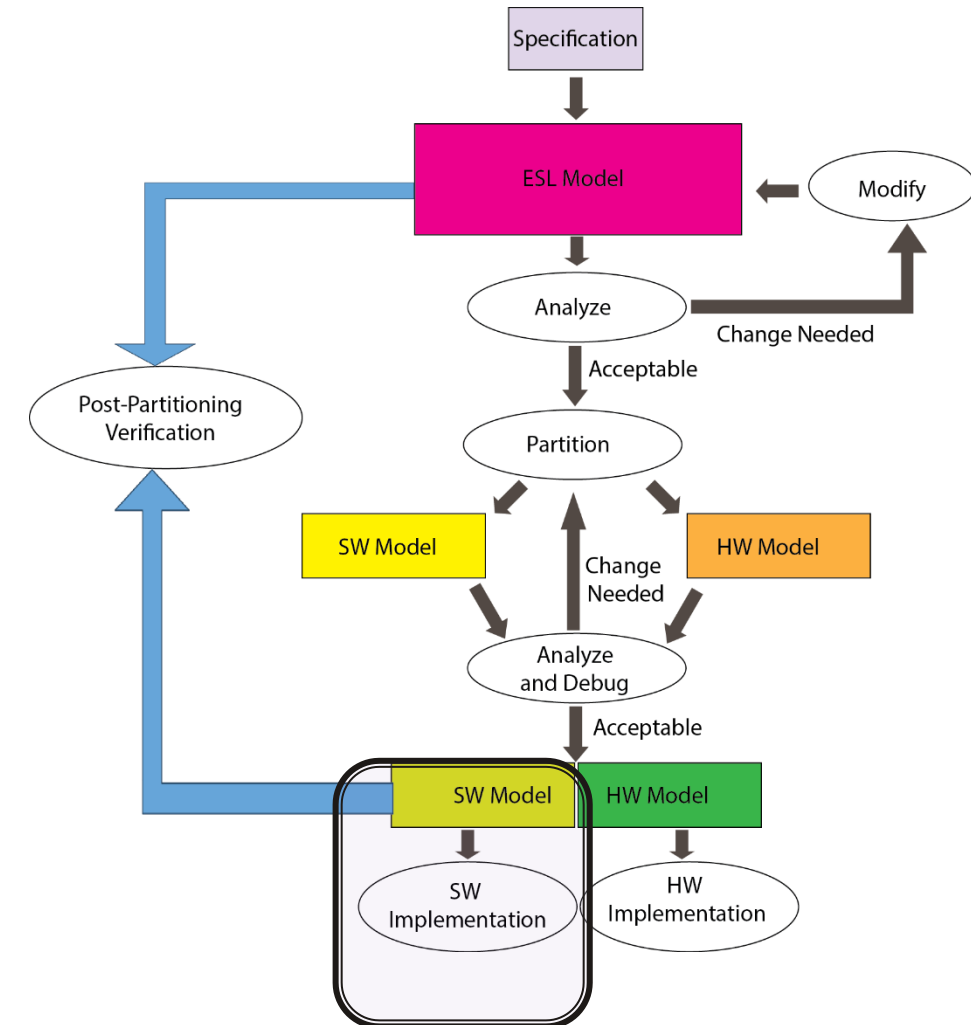
Software Implementation (1)

Evolution of Software Development

- Without ESL (Earlier) :
 - Software developed after the hardware was designed and implemented
- With ESL:
 - Software developed for the current hardware implementation
 - Software is further refined as the hardware model is refined

Benefits of ESL

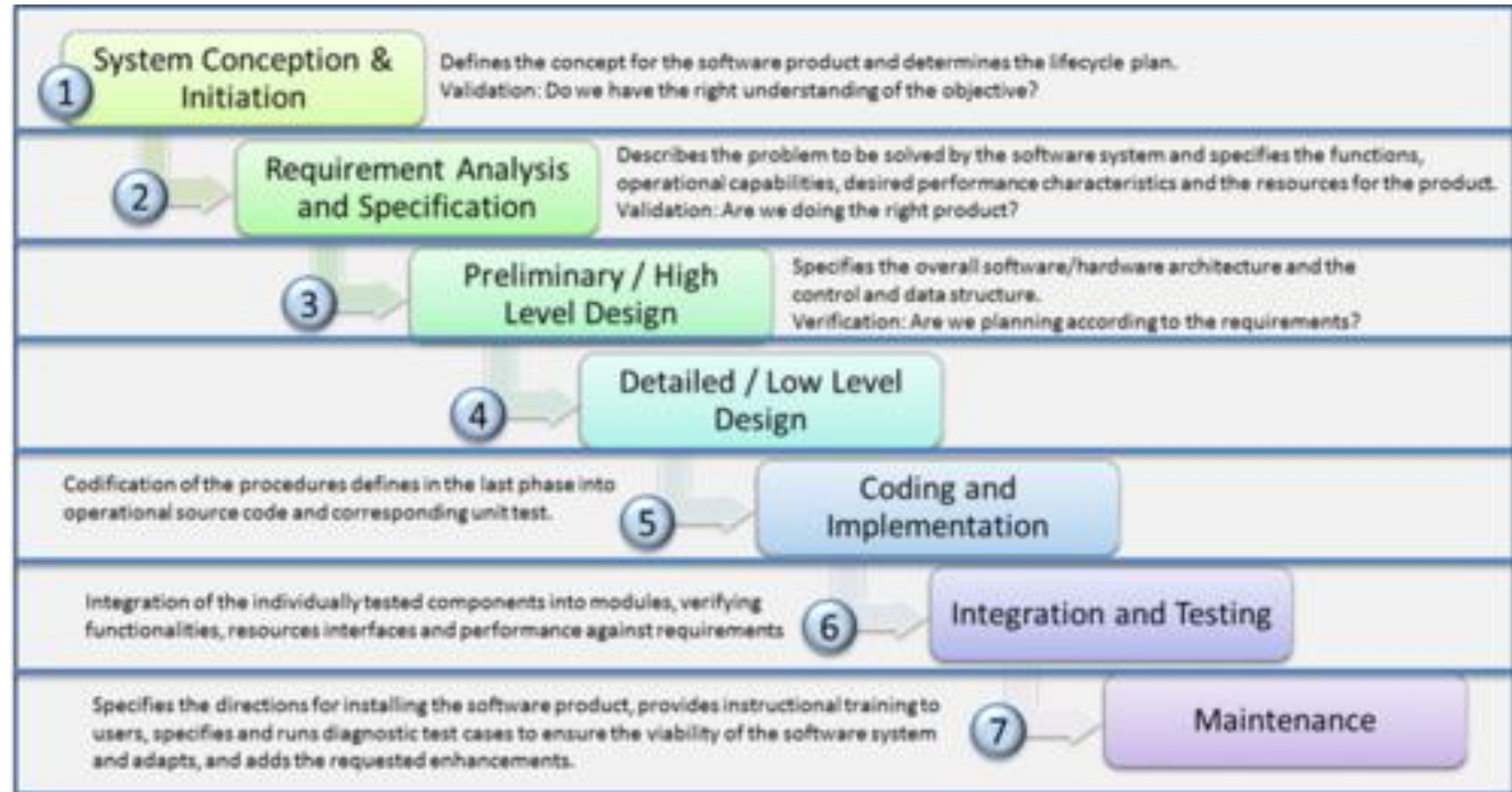
- Quicker development of the system (software and hardware development becomes concurrent rather than serial)



Software Implementation: Waterfall Model

- Without ESL (Earlier): Waterfall Model

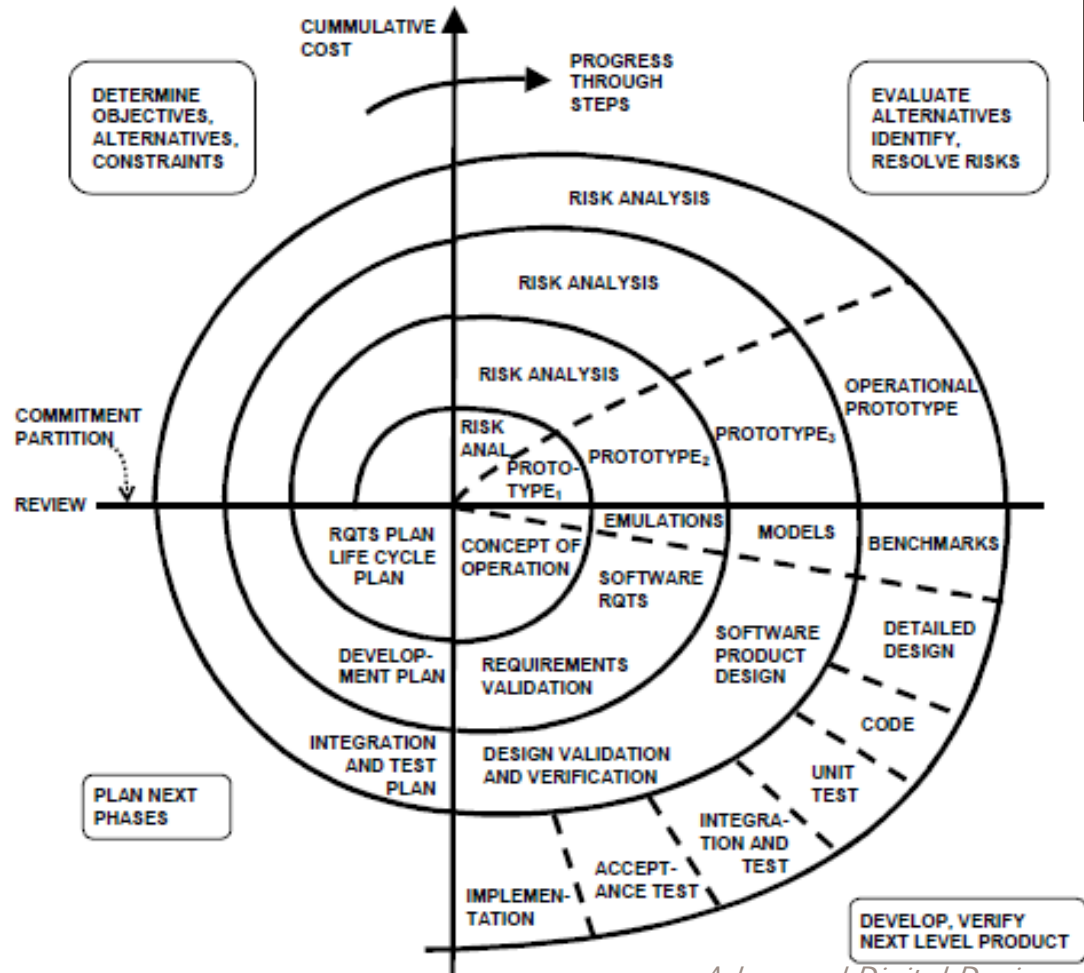
- Linear and sequential approach
- Output of one task is the input to the other



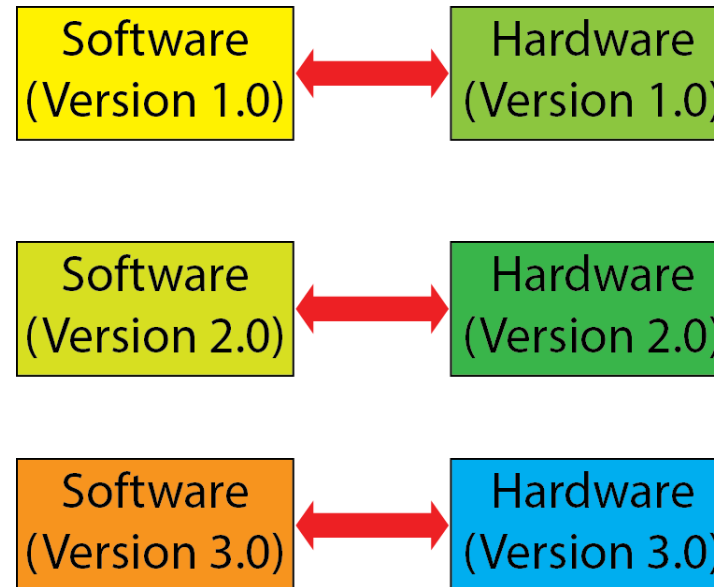
https://cio-wiki.org/wiki/Waterfall_Model

Software Implementation: Spiral Model

- With ESL: Spiral Model



- Iterative approach to software development
- Flexibility
- Works in the evolutionary method

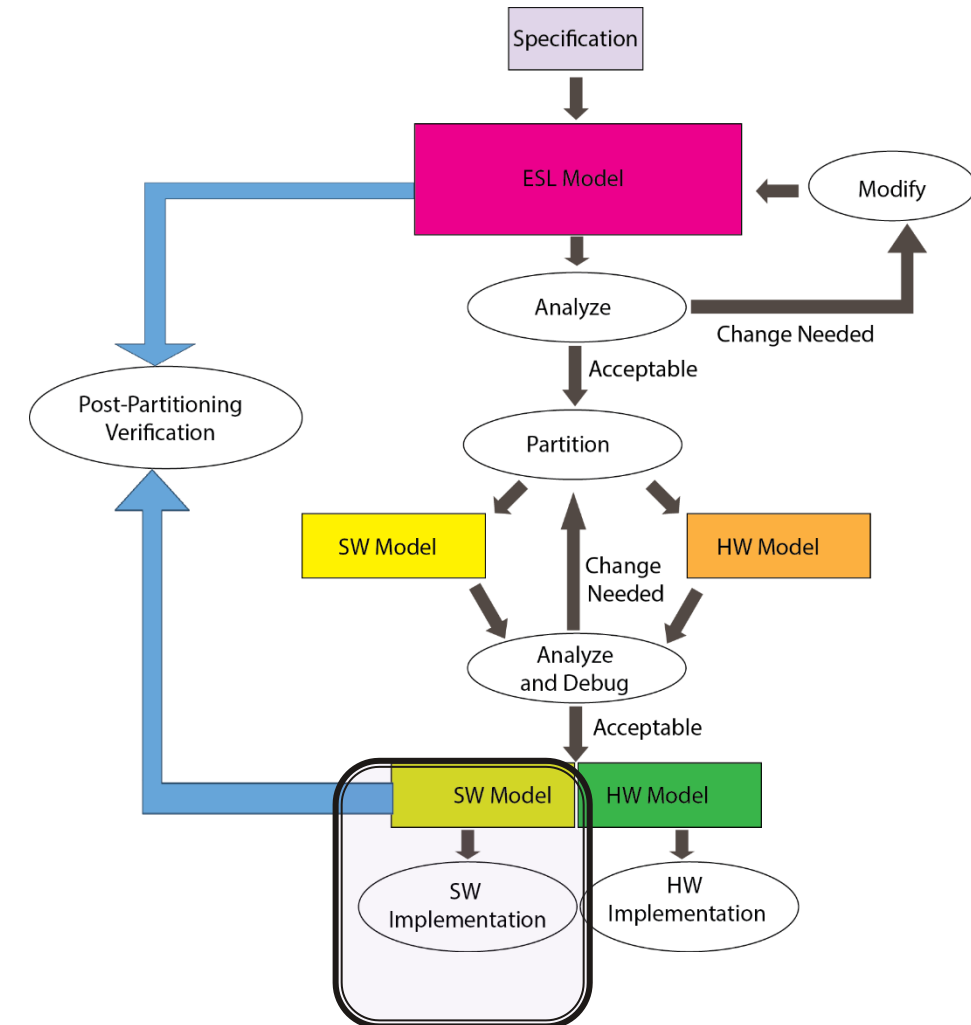


Boehm, B (July 2000). "Spiral Development: Experience, Principles, and Refinements" (PDF). Special Report. Software Engineering Institute. CMU/SEI-2000-SR-008.

Software Implementation: Application of ESL (1)

Create executable software using ESL models (or its refined version)

- Can be manually done
- Can generate code automatically from a high-level model such as UML, or SysML: **software synthesis** (analogous to logic synthesis)
- Developing debuggability a key concern: bug can be in the implemented software, or it can be in the environment (rest of the system)



Software Implementation: Application of ESL (2)

ESL models provide a run-time environment for developing executable software

- ESL model helps gain observability in the hardware

Scope can be:

- ISS (Instruction Set Simulator) in which only CPU's registers and memory are visible
- Virtual platform: the entire system is visible including the user interface, mass storage, hardware accelerators

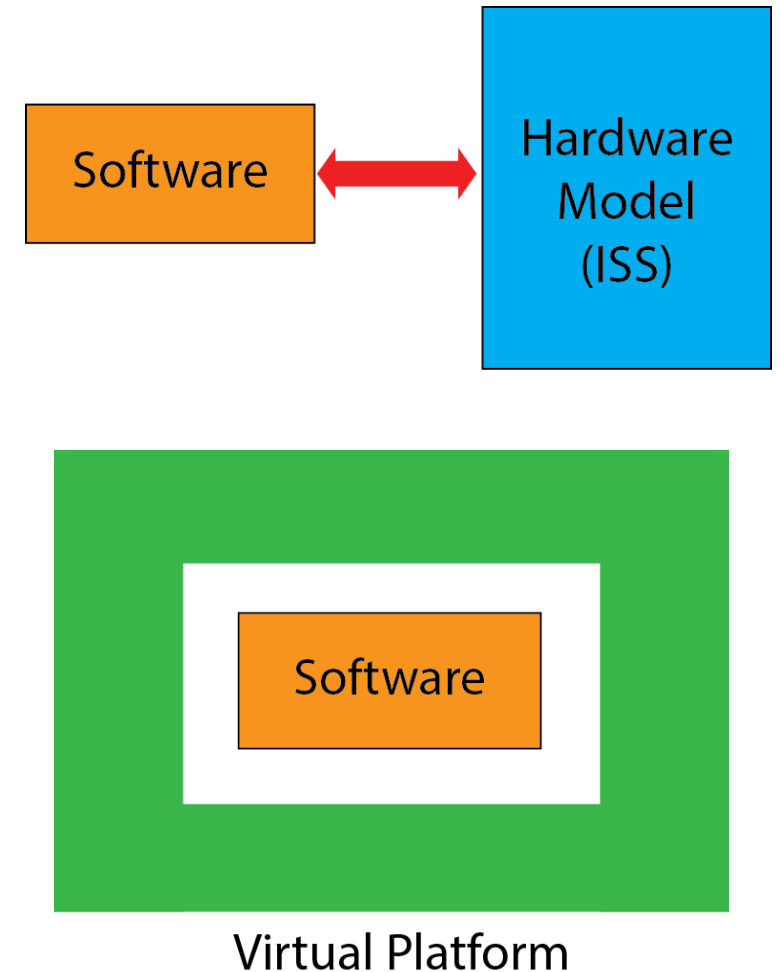
Time domain accuracy can be:

- untimed, cycle-accurate, RTL

Execution platforms can be:

- normal workstation, FPGA-based SoC emulation platforms

- Measuring performance: speed and memory footprint



Software Implementation: Debuggability

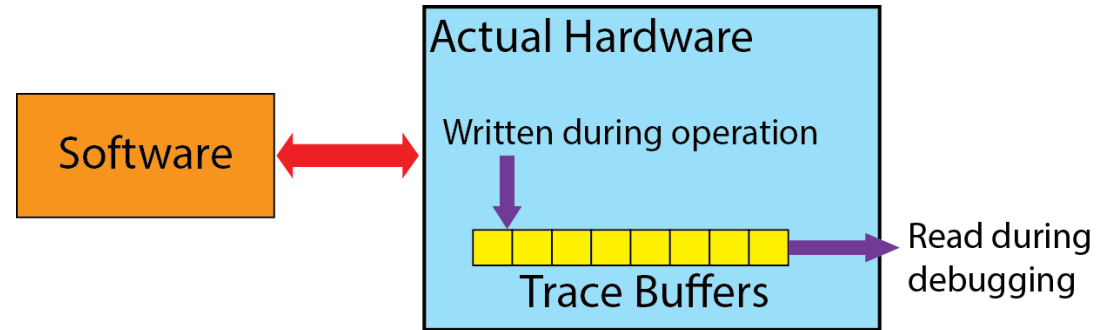
- ESL has advantages in debuggability for a system

Hardware-based

- Hardware can provide trace buffers for debuggability, but has disadvantages
 - Hardware cost (area, performance)
 - Security issues

ESL-based

- ESL models can provide debug information while developing software
 - Program counter and data trace can be obtained from such models
 - No hardware cost
 - No security issues



Hardware Implementation (1)

Hardware Implementation

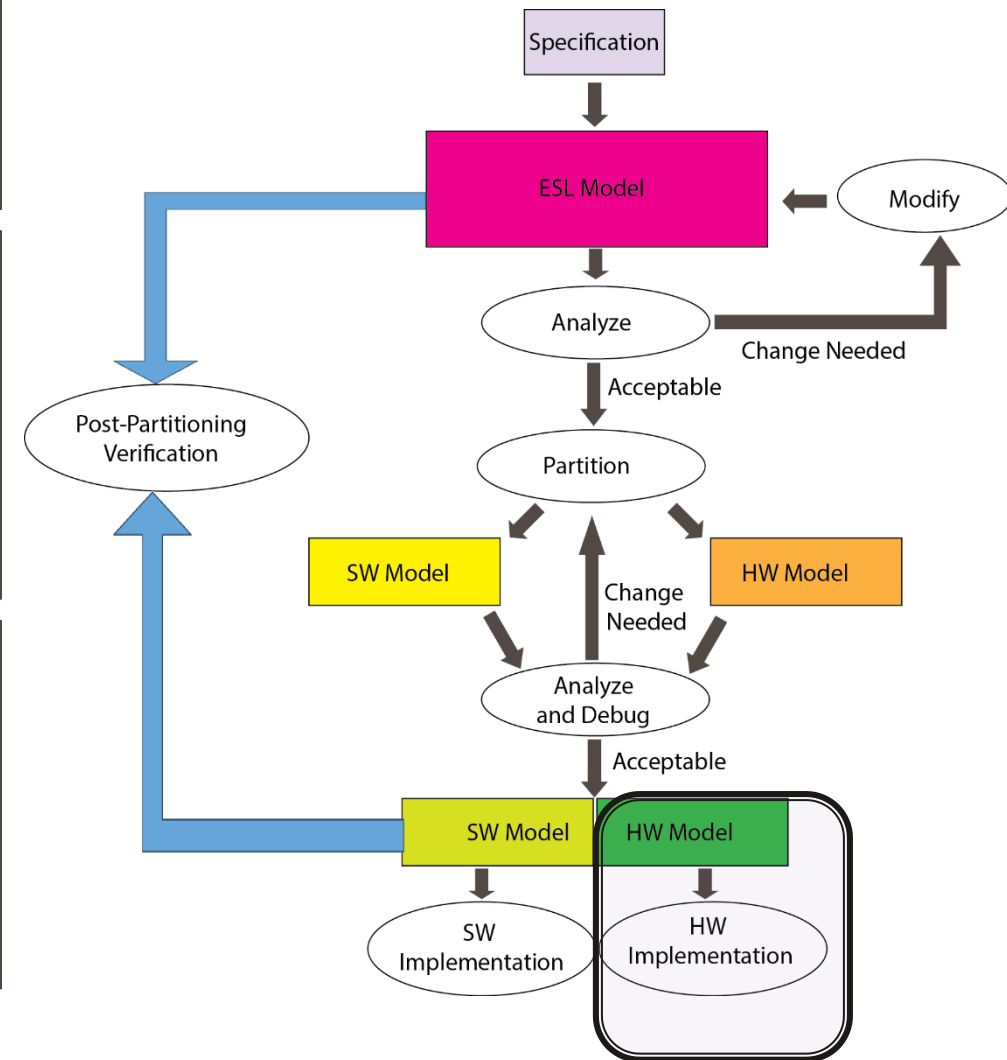
- Creating models (RTL) that may be synthesized into gate-level models.

Implementation options

1. Extensible processors
2. Co-processors
3. ASIC
4. FPGA

Extensible processors

- Customized for the application by the addition of SIMD instructions, MAC units, multioperation instructions, etc.
- Vendor provides tools/flows to convert instruction definition to RTL



Hardware Implementation (2)

Co-processors

- Can be DSP, customized VLIW, etc.
- Attached to the general-purpose processor through bus and mapped to its memory/IO space
- Vendors provide tools to generate co-processors for a given algorithm

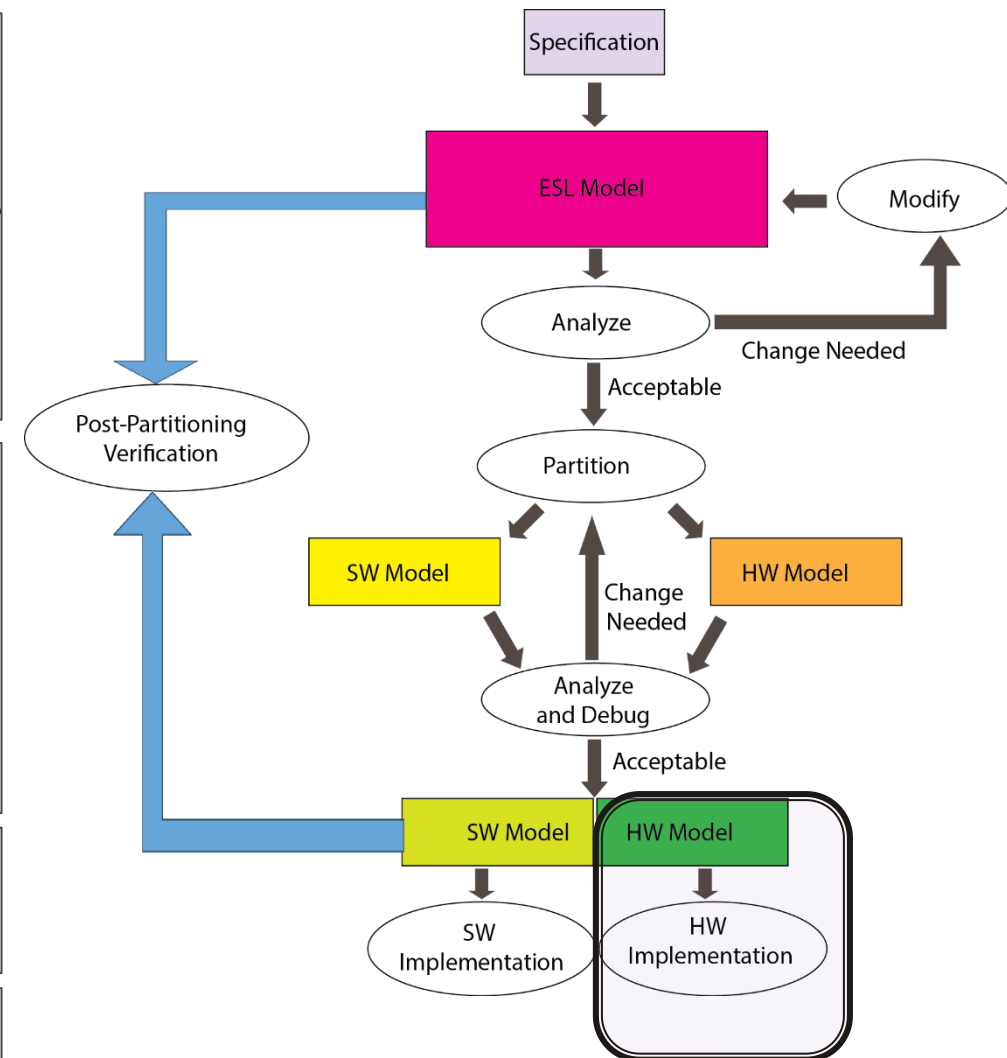
ASIC/FPGA

- High-level synthesis (HLS)
 - From C, C++ SystemC to RTL
- Enables more productive hardware design
- Exploration of scheduling, resource allocation, etc.

IP Assembly

- Composition of SoC by integrating IPs

We will discuss High-level synthesis and IP assembly later



Implementation Verification

Verify that design intent has been preserved in the implementation

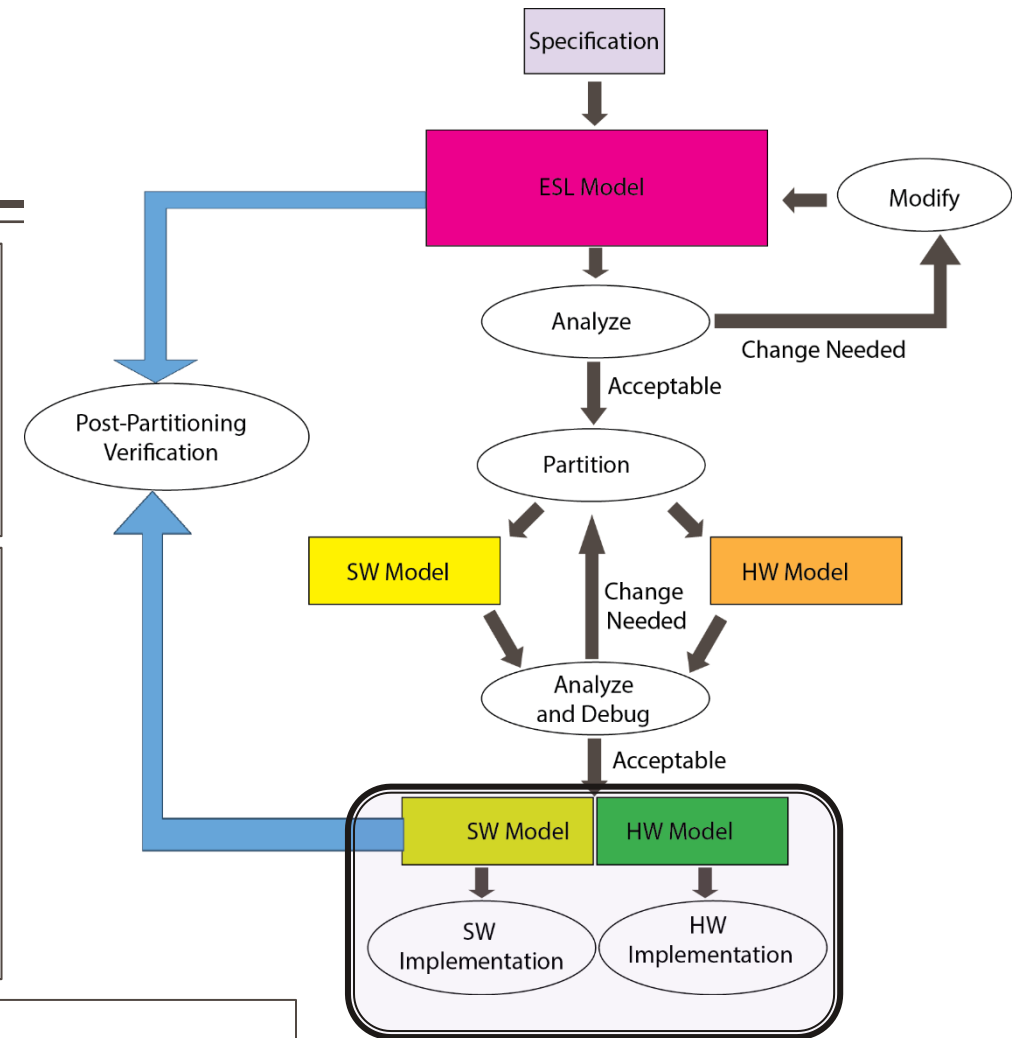
- Implementation: software and RTL
- Reference: post-partition model

Speed Limitations

- Verify at the subsystem level to avoid speed limitation
- Use mixed abstraction levels for verification
- Use hardware emulators: FPGA prototypes
- Use only critical testcases at system level

Types of Verification

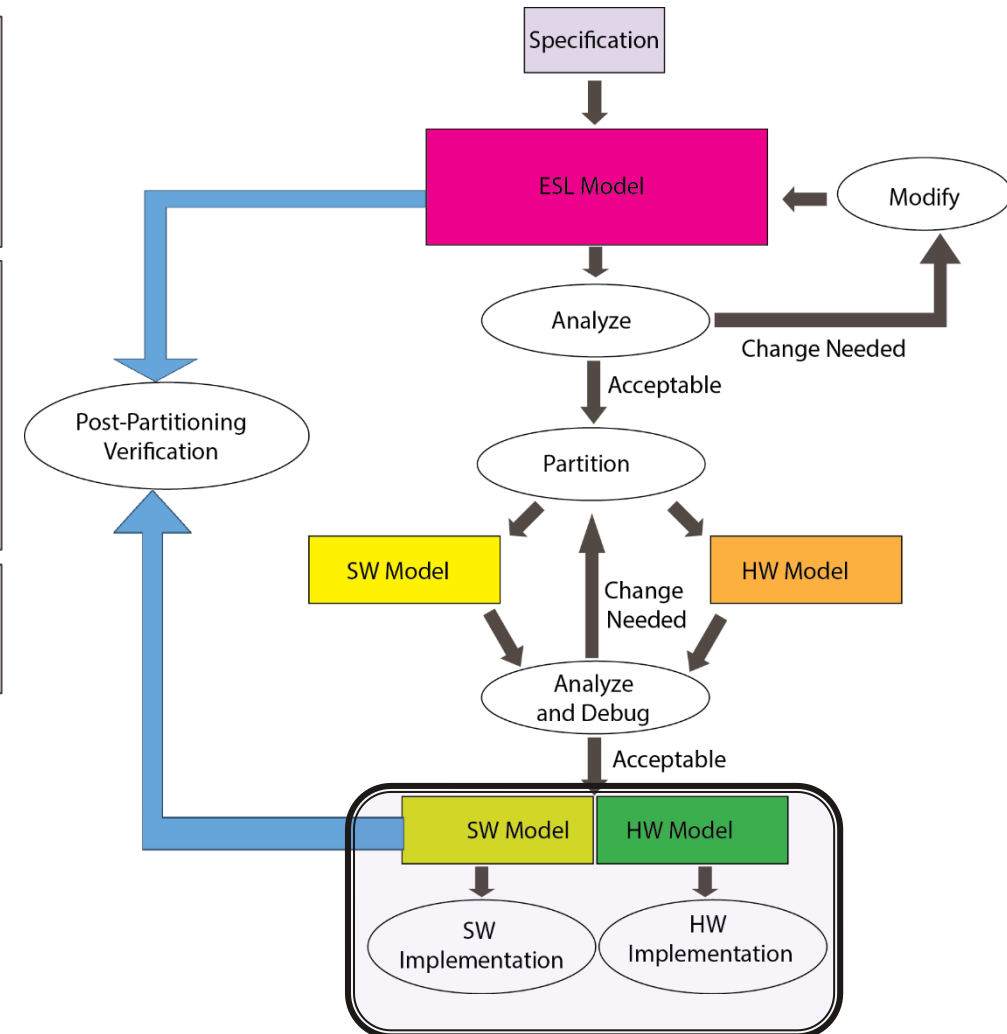
- Positive: design satisfies a requirement (example: valid inputs)
- Negative: no flaw exist in the implementation (example: invalid inputs)



Verification IPs

- Verification software written with reuse in mind is referred to as Verification IP (VIP)
- Reuse: requires configuration and instantiation
- Two types
 - **Dynamic VIP:** stimulus generation, response checking, coverage measurement
 - **Static VIP:** assertion library or properties

We will discuss Advanced Simulation Concepts and Assertions in later lectures



SystemC

History and Versions

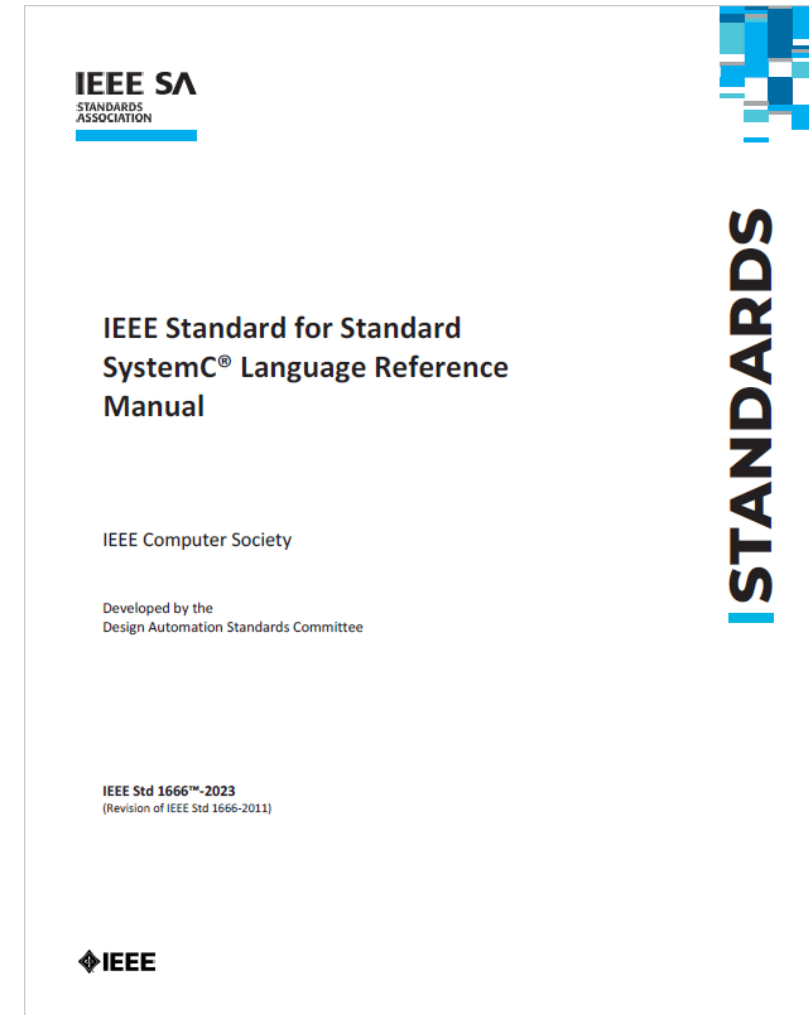
History

- The development of SystemC as a standard for modeling hardware started in 1996.
- Version 2.0 of the proposed standard was released by the **Open SystemC Initiative (OSCI)** in 2002.
- In 2006, SystemC became IEEE Standard 1666-2005



SystemC community portal: <https://systemc.org/>

- The SystemC reference implementation version 3.0.0 is now available
- You can download SystemC at GitHub:
<https://github.com/accellera-official/systemc/releases/tag/3.0.0>



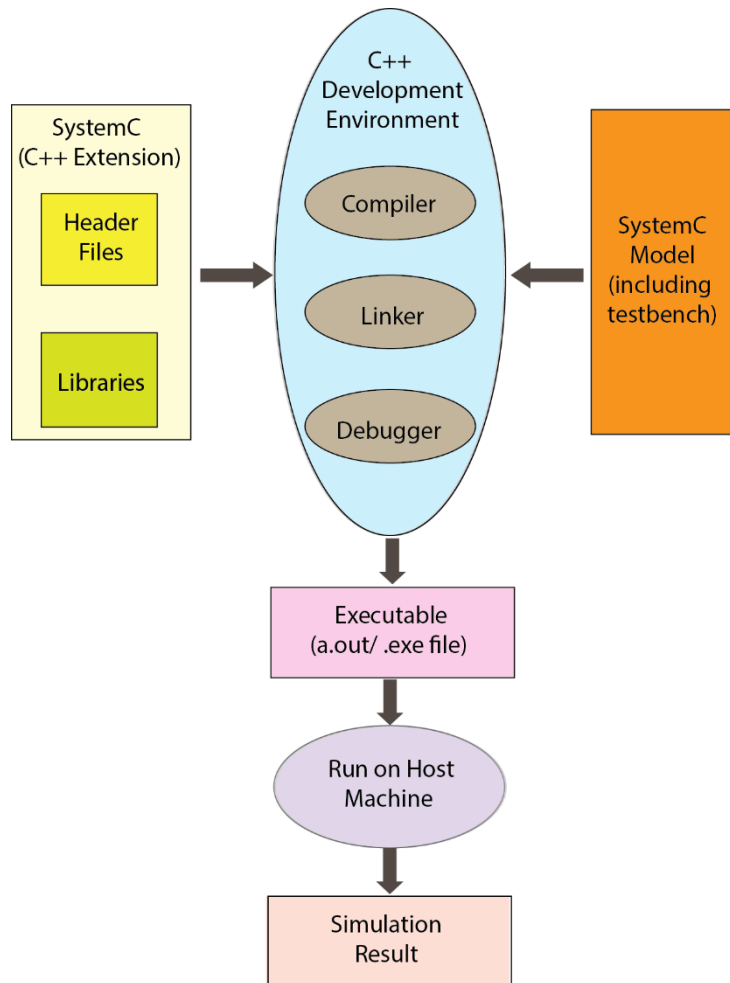
Need of SystemC

What is the need?

- SystemC bridges the divide between the worlds of hardware and software
 - To model the parallelism of hardware, in a mainstream programming language
-
- SystemC is based on the existing C++ language
 - Super-set of C++
 - Any C++ program is automatically a valid SystemC program.
 - SystemC uses the template, macro, and library features of C++ to extend the language.

```
1  #include <systemc>
2  #include "fifo.h"
3  #include "producer.h"
4  #include "consumer.h"
5
6  class top : public sc_core::sc_module
7  {
8      public:
9          fifo fifo_inst{"Fifo1"};
10         producer prod_inst{"Producer1"};
11         consumer cons_inst{"Consumer1"};
12
13         top(sc_core::sc_module_name name)
14         : sc_core::sc_module(name)
15         {
16             prod_inst.out(fifo_inst);
17             cons_inst.in(fifo_inst);
18         }
19     };
20
21     int sc_main (int, char *[]) {
22         top top1("Top1");
23         sc_core::sc_start();
24         return 0;
25     }
```

How to do simulation?



- Any C++ development environment can be used
- System Libraries and Header Files
 - Freely downloadable
- Compile the SystemC model (along with test bench), link, and get the executable
- Run on the host (UNIX/Windows)
 - Simulation results generated

SystemC: Hello World Program

```
#include <systemc.h>

SC_MODULE(Hello_SystemC) { // declare module class

    SC_CTOR(Hello_SystemC) { // create a constructor
        SC_THREAD(module_thread); // register the process
    }

    void module_thread(void) { // define the process registered
        SC_REPORT_INFO("Module Thread Saying:", " Hello SystemC World!");
    }

};

int sc_main(int sc_argc, char* sc_argv[]) {

    //create an instance of the SystemC module
    Hello_SystemC HelloWorld_i("HelloWorld_i");

    sc_start(); // invoke the simulator

    return 0;

}
```

Code in a file main.cpp

Module `Hello_SystemC`:

- Constructor is defined
 - A thread named `module_thread` is registered
- Thread `module_thread` is defined
 - Prints "Hello ..."

`sc_main` is the starting point:

- Instance named `HelloWorld_i` created
- Simulation started

Hello World: Compiling

Environment

- Using shell: tcsh (other shells can also be used)
- Go to the directory where code exists

Compilation

- Compiler: `g++`
- Include file path:
 - `-I<path_of_include_files>`
- Path of SystemC libraries:
 - `-L<path_of_libraries>`
- Name of SystemC library to link:
 - `-lsystemc`
- Source code: `main.cpp`
- Output: `a.out`

```
sneh@DESKTOP-C6Q06CF:~/test/systemC/hello_world$ tcsh
DESKTOP-C6Q06CF:~/test/systemC/hello_world> ls
main.cpp  readme.txt
DESKTOP-C6Q06CF:~/test/systemC/hello_world> g++ -I/home/sneh/systemC/systemc-master/include -L/home/sneh/systemC/systemc-master/lib-linux64/ -lsystemc main.cpp
DESKTOP-C6Q06CF:~/test/systemC/hello_world> ls -lrt
total 36
-rw-r--r-- 1 sneh sneh  188 May 23 16:44 readme.txt
-rw-r--r-- 1 sneh sneh   508 Jul 21 13:28 main.cpp
-rwxr-xr-x 1 sneh sneh 26848 Jul 21 13:36 a.out
```

```
DESKTOP-C6Q06CF:~/test/systemC/hello_world> ./a.out
./a.out: error while loading shared libraries: libsystemc-3.0.0.so: cannot open shared object file: No such file or directory
```

References

- Martin, Grant, Brian Bailey, and Andrew Piziali. ESL design and verification: a prescription for electronic system level methodology. *Elsevier*, 2010.
- Saurabh, S. (2023). Introduction to VLSI Design Flow. Cambridge: *Cambridge University Press*.